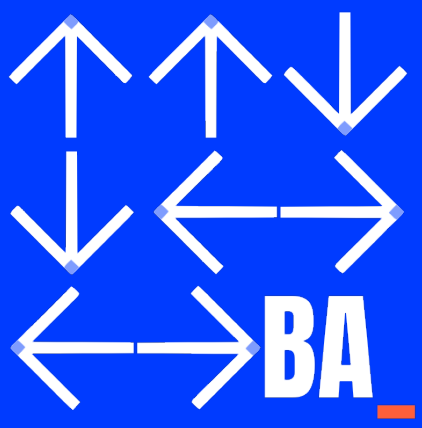
{EPITECH}

BOOTSTRAP_

< GAMEBOY PROGRAMMING />



BOOTSTRAP

Introduction

> “Nintendo is always one step behind.” “Nintendo consoles are not technically advanced; they’re for children.” ...

Nintendo is the richest Japanese company, and this is precisely because it uses outdated technologies. Its core philosophy was expressed by Gunpei Yokoi, a technical genius at Nintendo, as “lateral thinking with withered technology.”

In other words: how to create market value using technically outdated hardware. Technical constraints and frugality force creativity, and allows them to make sure that their products are profitable from the very first sale.

In this project, you'll have to be creative and clever. Follow in Nintendo's footsteps, and do something great.

> As usual, this bootstrap is separated in exercises, feel free to do them (or not) in any order

Emulator - Retroarch

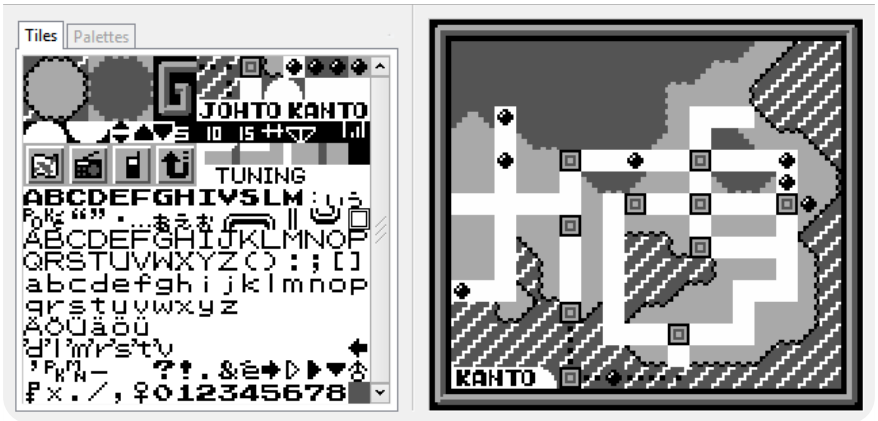
This project will be tested on the Retroarch Emulator, with the preset core Gambatte. It is cross-platform and open-source, with a good set of features and options.
It should be easy enough to install and to run on your computer. However, using an emulator to play games is one thing; using it to program games is another.Therefor, you might want to take some time to search around for other solutions. The two aspects an emulator must fulfill to allow an enjoyable programming experience are:
- Debugging tools
- Good accuracy
How does Retroarch compare to the alternatives you’re able to find? Do you think it’s better if you and the rest of your group work with the same tools or with a different one? Are some emulators good for parts of the project and other good at others?

Graphics - Your First Splash Screen

Setup

This part of the Bootstrap will be done using **RGBDS**.

You will use the code from this tutorial



As you see, the first step in creating graphics for the gameboy is hardcoding the **tiles**. Tiles are 8x8 squares of pixels. Sprites and Backgrounds are also hardcoded arrays, but instead of being composed of pixels they are composed of references to the tilemap.

> There's also the Window, but you'll worry about that later

Tasks

The first step of this exercise is to find an asset editor, and use it to modify the written text. Instead of Hello World, try writing anything else.

> You could try and do it by hand, but this is not the spirit of this exercise. You should find a good asset editor that will help you gain time in the future

🎨 The next step is **Palette Animation**. Tiles on the gameboy are composed of 4 values and the gameboy can show 4 colors but those don't have to correspond. Instead, a tile assigns **color indices** to each of its pixels, ranging from 0 to 3. When a tile is used in the Background or Window, these color indices are associated with a palette. When a tile is used in an object, the indices 1 to 3 are associated with a palette, but ID 0 means transparent.

This means you can choose to have all values correspond to the same color. Find a cool way to make your splash screen appear and disappear by using palette value swapping. Adjust the timing, create an intro and an outro.

🌟 If you feel daring, add or use sprites and look into the additional flags that can be toggled :
- Priority : Should the sprites appear behind the background and window?
- Vertical Flip : Should we flip our sprite's tile vertically?
- Horizontal Flip : Should we flip our sprite's tile horizontally?

Background Scrolling and Input

Setup

This part of the Bootstrap will be done using GBDK 2020.

> 📢 Be careful. It might be tempting to use C for this project and use the familiar syntax that GBDK offers, but some optimisations will be harder to do, and those **are** rewarded in the grading.

Use and familiarize yourself with the code from this tutorial.

Tasks

☹️ Now, this little guy looks lonely and lost. He has nowhere to go.
Your job is to create a background for him, using any tile you want. Then, when the guy reaches the end of the screen, scroll the background and make it move. The background should scroll infinitely.

🍎 Next step: Add an apple or another collectable to the **background** layer! Then, try and make your sprite happy when it touches the apple.
There is no real collision happening, you have to compute the apple's position relative to the sprite by your own means.

v 0.1

{EPITECH}